

Guia do Desenvolvedor — ERP Cortexis Tech · Controlo de Stock

1. Objetivo e visão de conjunto

Aplicação desktop offline-first para controlo de stock. O processo Electron contém as regras críticas, a base SQLite cifrada e as integrações com o sistema operativo; o processo React apenas apresenta a interface. Não há acesso direto do browser à base de dados.

Fluxo principal: ****React (renderer) → `window.estoqueApi` (preload) → IPC validado → `electron/main.ts` → `electron/db.ts` (fachada) → repositórios/serviços → SQLite cifrada****.

`db.ts` é uma fachada pública durante a migração para módulos menores. Não introduzir novas regras volumosas nesse ficheiro: criar um repositório ou serviço em `electron/database/` e expor apenas a função necessária na fachada.

2. Arranque local

Pré-requisitos: Node 20+, npm 10+ e macOS/Windows/Linux suportado pelo Electron.

```
```bash
npm install
env -u ELECTRON_RUN_AS_NODE npm run dev # necessário neste ambiente macOS quando a variável existir
```
```

Para interface sem Electron, execute `npm run dev:web`; neste modo os dados ficam em memória, backups, atualização e segurança do cofre não representam produção. Credenciais iniciais: `admin` / `admin123`; a troca da palavra-passe é obrigatória.

Verificações antes de entregar alterações:

```
```bash
npm run typecheck
npm test
npm run build
npm run smoke
npm run test:e2e
```
```

3. Pastas e ficheiros da raiz

| Caminho | Responsabilidade |
|-------------------------------|--|
| `src/` | Interface React, páginas, componentes e API de fallback para modo web. |
| `electron/` | Processo principal, IPC, base de dados, licença, atualização e telemetria. |
| `shared/` | Tipos, contratos IPC, rótulos e utilitários partilhados entre renderer e main. |
| `public/` | Favicon, ícones web e inicialização do tema antes do React. |
| `build/` | Ícones de empacotamento e entitlements macOS. |
| `scripts/` | Automação de ícones, manual, chaves, licenças, smoke e assinatura. |
| `license-generator/` | Aplicação Electron separada para emissão de licenças assinadas. |
| `docs/` | Documentação de produto, operação, segurança e engenharia. |
| `e2e/` | Cenários Playwright da interface. |
| `.github/workflows/` | CI e criação de releases. |
| `dist/`, `dist-electron/` | Saídas regeneráveis de `npm run build`; não editar. |
| `release/` | DMG/ZIP/instaladores e metadados de atualização; não é código-fonte. |
| `package.json` | Scripts e dependências do aplicativo principal. |
| `vite.config.ts` | Vite, React e compilação do Electron/preload. |
| `electron-builder.config.cjs` | ID da aplicação, conteúdos do pacote, targets e assinatura. |
| `playwright.config.ts` | Configuração E2E. |
| `tsconfig*.json` | Regras TypeScript para renderer e ferramentas Node. |
| `.env.example` | Variáveis opcionais, como Sentry. Nunca guardar segredos em `.env` versionado. |

4. Renderer React (`src/`)

| Caminho | O que faz |
|------------|-----------------------------------|
| `main.tsx` | Monta React e os estilos globais. |

| `App.tsx` | Rotas, inicialização, estado de licença/autenticação e bloqueios de acesso. |
 | `pages/` | Ecrãs: painel, produtos, categorias, fornecedores, clientes, compras, vendas, receitas, fabrico, movimentos, inventário, relatórios e configurações. |
 | `components/AppLayout.tsx` | Menu, título, sessão e permissões de navegação. |
 | `components/ModalForm.tsx` | Diálogo reutilizável e acessível para formulários. |
 | `components/CollectionPage.tsx` | Estrutura de listas/cadastros. |
 | `components/StatusBadge.tsx` | Estados visuais consistentes. |
 | `lib/api.ts` | Acesso tipado a `window.estoqueApi`; fornece implementação em memória no modo web/teste. |
 | `lib/auth.test.ts`, `stock.test.ts`, `backup-update.test.ts` | Testes do contrato de interface/API em memória. |
 | `lib/format.ts`, `units.ts`, `branding.ts`, `theme.tsx`, `toast.tsx` | Formatação, unidades, marca, tema e notificações. |
 | `styles.css` | Estilos globais e componentes. |

Regra: uma página nunca chama Electron, `fs` ou SQLite. Toda mutação deve chamar o contrato em `shared/api-contract.ts`, tratar o resultado com `unwrap` e atualizar o estado após sucesso.

5. Processo principal (`electron/`)

| Ficheiro | O que faz |
 |---|---|
 | `main.ts` | Cria janela segura, regista handlers IPC, aplica autorização admin/operador, inicia base, backup automático e atualizador. |
 | `preload.ts` | Expõe apenas a API allowlisted via `contextBridge`; é a fronteira de segurança renderer/main. |
 | `db.ts` | Fachada de operações de domínio e compatibilidade; delega para repositórios/serviços. |
 | `license.ts` | Identidade da instalação, validação Ed25519, cifragem local da chave e ativação. |
 | `updater.ts` | Verificação e instalação de atualizações nos builds empacotados. |
 | `telemetry.ts` | Sentry opcional e captura segura de erros. |
 | `db.test.ts`, `license.test.ts` | Integração SQLite e regras de licença. |

IPC e segurança

1. O renderer chama uma função declarada em `shared/api-contract.ts`.
2. `preload.ts` traduz para um canal IPC fixo.
3. `main.ts` valida o utilizador atual e exige administrador quando aplicável.
4. O handler chama a fachada da base e devolve `{ ok, data }` ou `{ ok: false, error }`.

Nunca exponha `ipcRenderer`, `require`, caminhos arbitrários ou métodos genéricos ao renderer. Para novo canal, atualizar contrato, preload, handler, fallback web e testes.

6. Base de dados (`electron/database/`)

| Área | Responsabilidade |
 |---|---|
 | `connection.ts` | Caminho da base, chave AES-256 no cofre do SO, SQLCipher, WAL, migração de base plana e `integrity\_check`. |
 | `types.ts` | Tipo interno da ligação SQLite. |
 | `audit.ts` | Contexto da operação e registo de auditoria. |
 | `migrations/001...008` | Evolução ordenada do esquema: base inicial, compras/fabrico, vendas, snapshots de custo, passwords, governação, inventário, unidades/lotos. |
 | `migrations/index.ts` | Executa migrations transacionalmente e preserva backup para rollback. |
 | `repositories/` | Leitura e mapeamento de produtos, compras, vendas, fabrico e utilizadores. |
 | `services/stock-service.ts` | Único ponto de atualização de saldo e criação de movimento. |
 | `services/costing-service.ts` | Custo médio ponderado e custo do produto acabado. |
 | `services/reversal-service.ts` | Estorno com motivo, validação de saldo e movimento inverso. |
 | `services/inventory-service.ts` | Sessão de contagem, referência congelada, submissão e aprovação. |
 | `services/unit-service.ts` | Conversão para unidade de stock com precisão controlada. |
 | `services/lot-service.ts` | Receção, rastreabilidade e seleção FIFO/FEFO. |
 | `services/backup-service.ts` | Backup WAL-aware, validação e retenção automática. |
 | `services/password-service.ts` | script, salt, política de palavra-passe e histórico. |
 | `services/diagnostics-service.ts` | Diagnóstico redigido para suporte; não inclui dados empresariais. |
 | `services/movement-query-service.ts` | Consulta filtrada do histórico de movimentos. |

Regras invioláveis

- Saldo não pode ficar negativo.
- Movimento confirmado não é apagado; correção é estorno/ajuste audível.

- Compras só recebem insumos; vendas só baixam produtos acabados.
- Fabrico consome receita e credita produto final na mesma transação.
- Uma migration nova deve ser aditiva, numerada e testada contra base anterior.
- Toda escrita crítica deve estar numa transação SQLite.

7. Fluxos funcionais

Entrada e custo

Fatura de compra → valida fornecedor/insumo → `stock-service` cria entrada → atualiza saldo → `costing-service` recalcula custo médio → auditoria.

Fabrico

Ordem confirmada → valida receita e disponibilidade → baixa cada insumo → calcula snapshots de custo → credita acabado → grava itens consumidos e auditoria numa transação.

Venda e estorno

Venda → valida produto acabado e stock → grava preço/custo snapshot → baixa stock. Estorno → exige motivo com pelo menos cinco caracteres → confirma que é reversível → cria entrada inversa e altera estado para `estornado`; o documento original permanece.

Inventário físico

Abrir sessão → congelar saldo de referência → registar contagens → submeter → administrador aprova → diferenças viram ajustes com origem `inventario\_fisico`.

8. Licenças e chaves

O produto incorpora somente `shared/license-public-key.ts`. A chave privada Ed25519 não pertence ao projeto, ao DMG ou ao Git. Armazene-a fora do repositório, por exemplo em:

```
```text
/Users/clemiltonsilva/.cortexis-license-vault/license-private-key.pem
```
```

Para CLI, defina `CORTEXIS\_LICENSE\_PRIVATE\_KEY` com esse caminho. O `license-generator/` solicita o PEM manualmente e confirma que a chave deriva a pública esperada antes de assinar. Licenças são vinculadas à identificação local da instalação e protegidas com `safeStorage`.

9. Build, pacote e release

- `npm run build`: TypeScript e bundles em `dist/` e `dist-electron/`.
- `npm run electron:build`: diretório de aplicação para inspeção.
- `npx electron-builder --config electron-builder.config.cjs --mac dmg zip`: DMG e ZIP macOS.
- `scripts/after-pack-macos.cjs`: verifica o pacote macOS.
- `scripts/ci-prepare-signing.mjs`: prepara segredos de assinatura no CI.
- `.github/workflows/ci.yml`: testes, smoke, build e pacote de validação.
- `.github/workflows/release.yml`: releases por tag `vX.Y.Z`.

Build não assinado serve apenas para QA. Distribuição comercial exige assinatura/notarização macOS e assinatura Windows conforme `GUIA-DE-ASSINATURA-E-NOTARIZACAO.md`.

10. Diagnóstico, backup e incidentes

Configurações (admin) mostra versão, esquema da base, integridade, última cópia, disco livre e erros redigidos. O pacote de suporte é JSON técnico, sem base de dados, chaves, palavras-passe, clientes ou valores.

Para incidente: não alterar a `.db` diretamente; exportar cópia, gerar pacote de suporte, preservar os ficheiros e seguir `GUIA-DE-OPERACAO-E-RECUPERACAO.md`.

11. Como alterar com segurança

1. Atualize tipos em `shared/types.ts` e o contrato IPC se a UI precisar da operação.
2. Coloque SQL de consulta em repositório e regra mutável em serviço.

3. Adicione auditoria e transação a mutações relevantes.
4. Acrescente migration apenas para mudança de esquema.
5. Atualize API web de fallback para que testes e preview continuem úteis.
6. Crie/ajuste teste unitário, integração SQLite e E2E conforme o risco.
7. Execute typecheck, testes e build antes de entregar.

12. Ficheiros que não devem ser editados manualmente

- `dist/`, `dist-electron/` e `release/`: gerados.
- `package-lock.json`: atualizado pelo npm, não manualmente.
- `build/icon.\*` e `public/favicon\*`: regenerar com `npm run icons`.
- `shared/license-public-key.ts`: mudar apenas quando se cria um novo par de chaves, o que invalida o modelo de licenças anterior.

13. Escopo atual

O módulo de vendas controla saída, recibo, preço e margem interna. Não é, por enquanto, um módulo fiscal português certificado. Não adicionar SAF-T, ATCUD, QR Code, comunicação AT ou alegações de conformidade fiscal sem decisão jurídica e produto específica.